

Question 1

Answer saved

Marked out of
5.00 Flag question

Time left 0:31:5

1) To construct a valid Binary Search Tree, use the keys in the following order:

8, 4, 12, 2, 6, 10, 14, 1, 3

Drag&drop the keys into the tree skeleton below the box (beginning with 8 as the root, according the aforementioned sequence order).

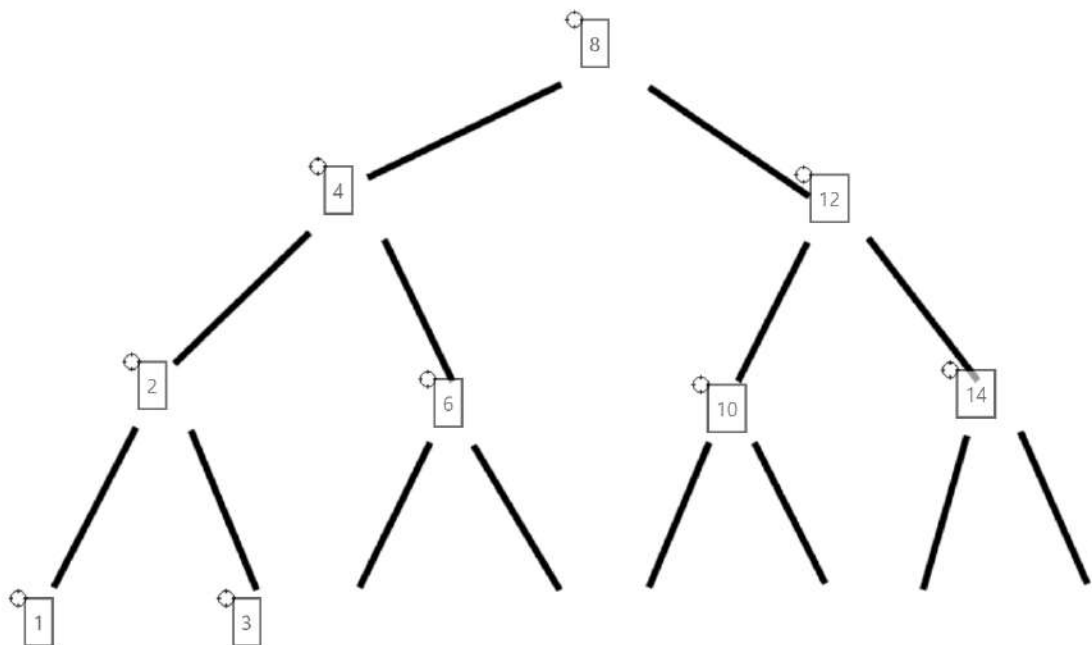
Note: Each key can be used only once and therefore some edges of the skeleton will remain unused.

Um einen gültigen binären Suchbaum zu konstruieren, verwenden Sie die Schlüssel in folgender Reihenfolge:

8, 4, 12, 2, 6, 10, 14, 1, 3

die Schlüssel in den Baum unterhalb der Box einfügen (beginnend mit 8 als Wurzel, gemäß der oben genannten Sequenz).

Achtung: Jeder Schlüssel kann nur einmal benutzt werden und daher können einige Kanten im Skelett ungenutzt bleiben



Question 2

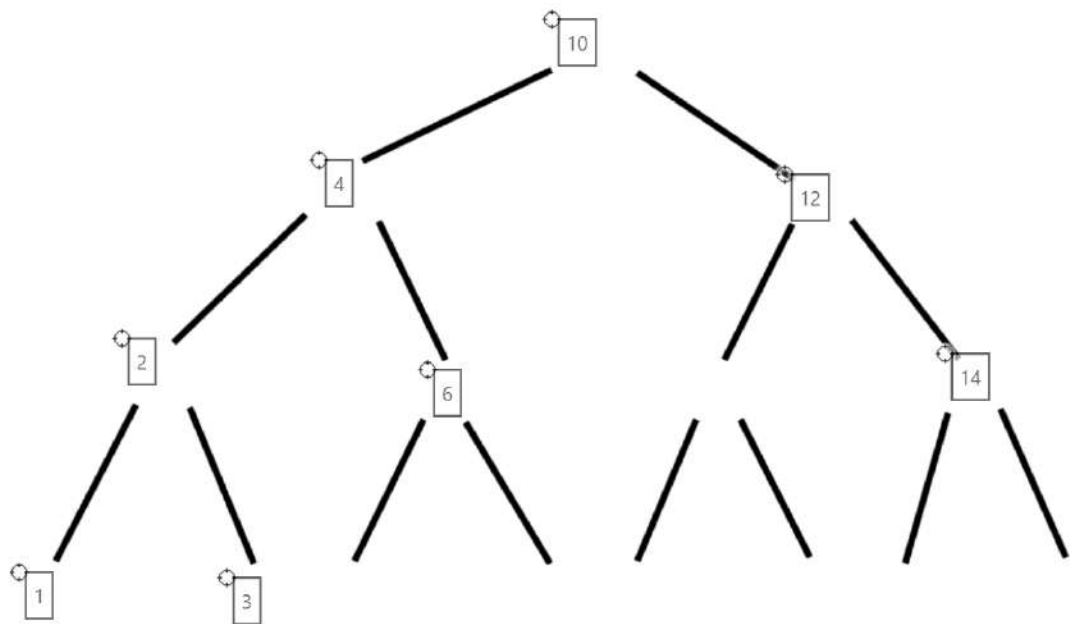
Answer saved

Marked out of
5.00

Flag question

2) Remove the root node (**with the key 8**) and provide the resulting Binary Search Tree.

Entfernen Sie den Wurzelknoten (mit dem Schlüssel 8) und geben Sie das resultierende Binärbaum an.



Question 3

Answer saved

Marked out of
5.00

 [Flag question](#)

Given the text and pattern below:

Text: YFZYFYZZFZZYZYZFYZFYZFYZFYZZFYZZZZZY

Pattern: FYZFYZ

To discover the pattern's initial instance in the text and calculate the number of comparisons, use the **Knuth-Morris-Pratt (KMP)** method. When the pattern is initially discovered, choose the appropriate amount of comparisons that the KMP algorithm makes.

Gegeben sind der Text und das Muster unten:

Text: YFZYFYZZFZZYZYZFYZFYZFYZFYZZFYZZZZZY

Pattern: FYZFYZ

Um die erste Instanz des Musters im Text zu finden und die Anzahl der Vergleiche zu berechnen, verwenden Sie die **Knuth-Morris-Pratt (KMP)** Methode.

Wenn das Muster zum ersten Mal entdeckt wird, wählen Sie die geeignete Anzahl der Vergleiche, die der KMP-Algorithmus durchführt.

- ☒ a. 6 comparisons
- ☐ b. 5 comparisons
- ☐ c. 4 comparisons
- ☐ d. 7 comparisons

[Clear my choice](#)

Question 4

Answer saved

Marked out of
5.00 Flag question

Construct the KMP failure function for the following search pattern

Erstellen Sie die KMP-Fehlerfunktion für das folgende Suchmuster.

ababcbabababcbabab

{punkte:TYP:=ANTWORT}

j	0	1	2	3	4	5	6	7	8	9	10	
P[j]	a	b	a	b	c	a	b	a	b	a	b	
f[j]	0	0	1	2	0	1	2	3	4	5	6	

j	11	12	13	14	15	16	17
P[j]	a	b	c	a	b	a	b
f[j]	0	1	2	3	4	5	6

Question 5

Answer saved

Marked out of
10.00

[Flag question](#)

Create the last-function ***last[ch]*** according to the **Boyer-Moore** algorithm for the pattern "banana" and the following search text:

banananabanabana

Erstellen Sie die last- Funktion ***last[ch]*** gemäß dem **Boyer-Moore**-Algorithmus für das Muster

banana und den folgenden Suchtext:

banananabanabana

Edit View Insert Format Tools Table Help



ch		b	a	n															
last[ch]	0	5	4																

table > thead > tr > th

9 words

Question 6

Answer saved

Marked out of
10.00

[Flag question](#)

We are provided with the recurrence scheme below:

$$T(N) = 2 T(N/2) + N, N \geq 2$$

$$T(1) = 1$$

Time left 0:30:49

Use **the Master Theorem** to solve this recurrence problem, and note the key stages involved in identifying the right case and calculating the asymptotic runtime complexity.

Hint: The buttons that are indicated below assist in creating superscripts and subscripts. You have two options for writing the square root: $\sqrt{N/2}$ or $\text{sqrt}(N/2)$. When writing logarithms, use the notation $\log_b(N)$, where b is the base.

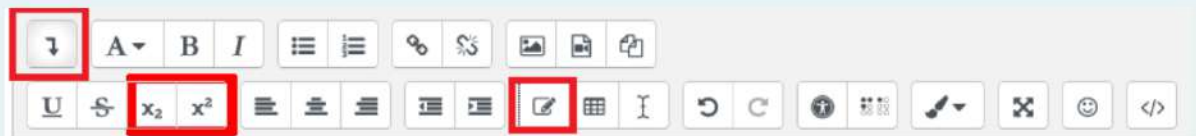
We sind mit dem untenstehenden Rekurrenz Schema versehen:

$$T(N) = 2 T(N/2) + N, N \geq 2$$

$$T(1) = 1$$

Verwenden Sie **den Master-Theorem**, um dieses Rekurrenzproblem zu lösen, und notieren Sie die wichtigsten Schritte zur Identifizierung des richtigen Falls und zur Berechnung der asymptotischen Laufzeitkomplexität.

Hinweis: Die unten angegebenen Schaltflächen helfen beim Erstellen von Hoch- und Tiefstellungen. Sie haben zwei Möglichkeiten, die Quadratwurzel zu schreiben: $\sqrt{N/2}$ oder $\text{sqrt}(N/2)$. Beim Schreiben von Logarithmen verwenden Sie die Notation $\log_b(N)$, wobei b die Basis ist.



Edit View Insert Format Tools Table Help

↶ ↷ **B** *I* A

1- case 3

2- $T(n) = \theta(n^{1.58} \cdot \log(n))$

3- with time complexity of $O(n)$

p

47 words

Question 7

Answer saved

Marked out of
10.00 Flag question

What is the time complexity for f of the following code?

Wie lautet die Zeit Komplexität für f des folgenden code?

```
p = 8
f = p
d = 0
i = 0

# Outer loop
for k in range(2, f+1):
    # Inner loop
    for j in range(1, k):
        d += j

# While loop with nested for loop
while i < f:
    for m in range(i, f):
        d += m
    i += 1

print("Final value of d:", d)
```

- ☐ a. $O(f)$
- ☒ b. $O(f^2)$
- ☐ c. $O(f \log(f))$
- ☐ d. $O(f*m)$

[Clear my choice](#)

Question 8

Answer saved

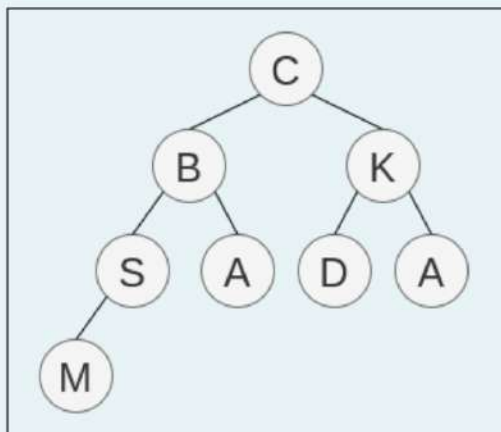
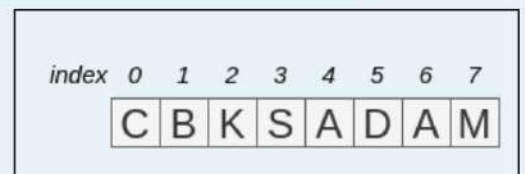
Marked out of 10.00

[Flag question](#)

Find the element at **index 9** in the following heap after executing **3 times** `removeMin()` and its necessary upheping for restoring the order and structure property of the following minheap:

Finden Sie das Element an Index 9 nach der Ausführung von 3-mal `removeMin()` und der notwendigen Umordnung zur Wiederherstellung der Ordnung und Struktur-Eigenschaft des folgenden Min-Heaps:

Index	0	1	2	3	4	5	6	7	8	9	10	11	12
	1	3	5	4	8	9	11	7	15	13	18	21	30

Heap represented as a **tree**Heap represented as an **array**.

- ☒ a. 13
- ☐ b. 18
- ☐ c. 11
- ☐ d. 15
- ☐ e. 9

[Clear my choice](#)

Question 9

Answer saved

Marked out of 10.00

[Flag question](#)

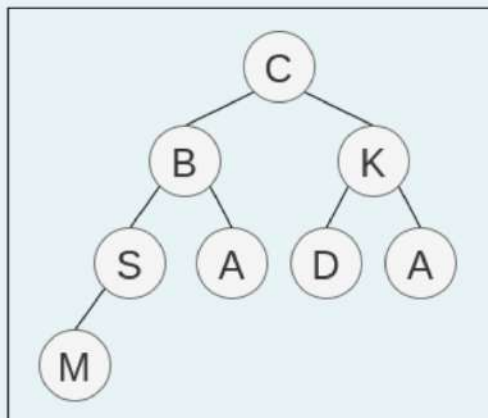
Time left 0:29:50

Starting from the **left side** and working your way down the provided series of integer values {39, 10, 1, 3, 19, 3, 1, 8, 6, 24, 31, 17, 25, 4, 23}, create a **MaxHeap** using the bottom-up building technique. Put it in the following table in array form (don't worry about showing every downheap movement in the array; just include every line once the heap is "heapified" once more).

Hint: The visual example that follows demonstrates how to transform a binary tree into an array. The root is represented by index position 0, while the left and right offspring of a node with index i are denoted by the indices $2*i+1$ and $2*i+2$, respectively.

Beginnend von der linken Seite und arbeitend Ihren Weg hinunter durch die bereitgestellte Reihe von ganzen Zahlen {39, 10, 1, 3, 19, 3, 1, 8, 6, 24, 31, 17, 25, 4, 23}, erstellen Sie einen Max-Heap mithilfe der Bottom-Up-Aufbautechnik. Geben Sie ihn in der folgenden Tabelle in Array-Form an (müssen Sie sich nicht um jede Downheap-Bewegung im Array kümmern; geben Sie einfach jede Zeile an, sobald der Heap einmal "geheapified" ist).

Hinweis: Die visuelle Beispiel, das folgt, zeigt, wie man einen Binärbaum in ein Array umwandelt. Die Wurzel wird durch die Indexposition 0 repräsentiert, während die linken und rechten Nachkommen eines Knotens mit Index i durch die Indizes $2i+1$ bzw. $2i+2$ bezeichnet werden.



Heap represented as a tree



index	0	1	2	3	4	5	6	7
	C	B	K	S	A	D	A	M

Heap represented as an array.

Edit View Insert Format Tools Table Help

↩ ↪ **B** *I* A ∨ 🔗 🔗

index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
0	39														
1	39	10													
2	39	10	1												
3	39	10	1	3											
4	39	19	1	3	10										
5	39	19	3	3	10	1									
6	39	19	3	3	10	1	1								
7	39	19	3	8	10	1	1	3							
8	39	19	3	8	10	1	1	3	6						
9	39	24	3	8	19	1	1	3	6	8					
10	39	31	3	8	24	1	1	3	6	10	19				
11	39	31	17	8	24	3	1	3	6	10	19	1			
12	39	31	25	8	24	17	1	3	6	10	19	1	3		
13	39	31	25	8	24	17	4	3	6	10	19	1	3	1	
14	39	31	25	8	24	17	23	3	6	10	19	1	3	1	4

Question 10

Answer saved

Marked out of
10.00

Flag question

Using the **BucketSort** method, sort the following array using base 5 keys {0, 1, 2, 3, 4} in ascending order.

[109, 2010, 11, 201, 10, 2, 421, 119, 51, 22, 1223, 55, 204, 1]

After every iteration (**but before the merging**), write down the contents **of the buckets** in the table below:

Verwenden Sie die BucketSort-Methode, um das folgende Array mit Basis-5-Schlüsseln {0, 1, 2, 3, 4} in aufsteigender Reihenfolge zu sortieren

[109, 2010, 11, 201, 10, 2, 421, 119, 51, 22, 1223, 55, 204, 1]

Nach jeder Iteration (**aber vor dem Zusammenführen**) schreiben Sie den Inhalt **der Eimer** in der untenstehenden Tabelle auf:

Edit View Insert Format Tools Table Help					
iteration 1	0->402	403->805	806->1208	1209->1611	1612->2013
iteration 2	109,11,201				
	10,2,119,51	421	empty	1223	2010
iteration 3	22,55,209,1				
	1,2,10,11,22	421	empty	1223	2010
iteration 4	51,55,109,119				
	201.204				
iteration 4	final result will be sorted in				
	one line, I will write it below as 4th iteration				
each bucket contains 402 elemnts					
**** 4th Iteration =					
1,2,10,11,22,51,55,109,119,201,204,421,1223,2010					

Question 11

Answer saved

Marked out of
10.00 Flag question

What's the output of the following recursive function when called with the input **n=6**?

Was ist die Ausgabe der folgenden rekursiven Funktion, wenn sie mit dem Eingabewert **n=6** aufgerufen wird?

```
def Rec_function:
    if n == 0:
        return 1
    elif n == 1:
        return 1
    elif n % 2 == 0:
        return n * Rec_function(n - 2)
    else:
        return n + Rec_function(n - 1)
result = complex_function(6)
print(result)
```

- ☐ a. 45
- ☐ b. 49
- ☐ c. 44
- ☐ d. 47
- ☒ e. 48

[Clear my choice](#)

Question 12

Answer saved

Marked out of
10.00 Flag question

Which of the following statements best describes the main difference between a queue and a stack?

Welche der folgenden Aussagen beschreibt am besten den Hauptunterschied zwischen einer Queue und einem Stack?

- ☐ a. A stack follows the First In, First Out (FIFO) principle, while a queue follows the Last In, First Out (LIFO) principle.
Ein Stack folgt dem First-In-First-Out (FIFO) Prinzip, während eine Queue dem Last-In-First-Out (LIFO) Prinzip folgt.
- ☐ b. A stack allows elements to be added at both the front and the back, while a queue only allows elements to be added at the front.
Ein Stack erlaubt das Hinzufügen von Elementen sowohl vorne als auch hinten, während eine Queue nur das Hinzufügen von Elementen vorne erlaubt.
- ☒ c. A stack allows elements to be added and removed only from the top, while a queue allows elements to be added at the back and removed from the front.
Ein Stack erlaubt das Hinzufügen und Entfernen von Elementen nur von oben, während eine Queue das Hinzufügen hinten und das Entfernen vorne erlaubt.
- ☐ d. A stack and a queue both allow elements to be added and removed from both ends.
Ein Stack und eine Queue erlauben es beiden, Elemente an beiden Enden hinzuzufügen und zu entfernen.

[Clear my choice](#)